Auditing Token Accounting and Tool Dispatch in an ARC-AGI-3 Agent

Response-to-action accounting, capped programs, and limits of feature comparisons

Jiayi Zhang | University of California, Berkeley

Abstract

An interactive agent generates both reasoning and executable tool calls, but only some replies issue environment actions. I audit development runs of one system (linked submission 57684318), with native baseline evidence, by partitioning completed generation, replaying capped calls and checking feature exposure. Across five populations, action-issuing responses carry approximately 46-57% of completed generation and terminal responses 2.6-7.9%; the partition reconciles usage, while native action history alone omits the terminal suffix. In three pre-guard comparison arms, capped replies consume 6-8% of generation. Among 26,484 responses, 29 expose capped Python payloads; eight parse and one issues an action. Feature comparisons remain inconclusive. These are accounting and dispatch findings, not established score gains.

Submitted system

ARC-AGI-3 asks an agent to learn unfamiliar interactive games from observations and actions rather than a supplied rulebook. A game contains successive levels. The agent must infer the controls, objectives and relevant state, then use what it learns to complete later levels. An episode is one attempt at a game; a model response is one completed generation request; an environment action is a move, click or other accepted game command. These units are not interchangeable.

Scoring rewards both level completion and action efficiency relative to a human baseline, with later levels weighted more heavily [1]. Internal reasoning and analysis-only tool calls consume compute but do not themselves add game actions. The notebook must also finish within a wall-clock ceiling. Consequently, minimizing game actions, reducing generation, and increasing serving throughput are different objectives; none is an adequate substitute for the official game score.

The system inherits Tufa Labs' Duck harness and Daniel Franzen's solution [2,3]. A large language model inspects rendered boards and sends Python to a persistent read-evaluate-print loop (REPL). Python can inspect observations, define helpers or call the environment's action interface. One model response can cause zero, one or several actions. The harness maintains conversation history, trims it at context boundaries and schedules multiple games against a shared local inference server. This structure motivates auditing the boundary between generated work and recorded actions.

Linked submission: guarded Test1 R2	Configuration
Identity	Submission **57684318**, exact notebook V1; public score **20.02**
Model/serving	Qwen3.8-Flash-Next W4A16; Pennyroyal SGLang; MTP decoding [3,4]
Sampling/resources	Temperature 0.7; ten admitted streams; RTX Pro 6000; nine-hour ceiling
Capacity	131,072 harness / 139,264 server tokens; 12,288 reply allowance
Test1 additions	Context/usage checks, bounded lifecycle, executable-model tools, capped-call guard; mechanics memory off

R0 is the hardened control; R1 adds mechanics memory, R2 executable-model tools, and planned R3 both. The public R0-R2 comparison predates the output guard; its R2 score is not the guarded submission's score. The added model interface supports defining, checking, planning with and acting through a proposed simulator, while direct actions remain available.

Separate candidate	My public score	Author-page score*
Native Franzen V1	26.39	33.68; best 34.30 V1
Native Fang V1 [7]	26.31	33.94 V1

*Historical screenshots; these are unmatched runs, not a variance estimate. Guarded R2 scored lower; neither improvement nor causal degradation is established. The separate 27.45 entry derives from Roman Tamrazov's notebook version 83 (V83), with deployment changes; it does not contain this guard.

Related work

AI Agents That Matter motivates cost-controlled evaluation and documents reproducibility problems [8]. HAL supplies standardized evaluation and log analysis; its Appendix A4.1 specifically reports charging full token prices for SWE-Agent despite cache hits [9]. This paper contributes a narrower response/action audit and dispatch counterexamples within one submitted agent. ReAct, Reflexion and Fang's solved-level memory precede the respective reasoning/memory ideas [5-7]; no priority claim is made.

Audit and rationale

Terms. A completed response has returned token usage. A response is **capped** when its finish reason is length; in these logs, every such response reaches the configured 12,288-token output allowance. **Charging** means adding returned completion tokens to the agent's cumulative usage counter, not assigning a dollar price. The **terminal suffix** comprises completed responses after the episode's last recorded action; an episode with no actions has only a terminal suffix. Unanswered requests remain unknown work.

Alignment. Within each game/pass, join transcript response-metadata blocks to usage-log response events by ordinal position, verifying equal counts and finish reasons. Test1 uses its token traces; native Franzen uses game-named request logs. Join the episode's action history by that same game/pass identity. Repeated action numbers and within-turn request indices are not globally unique keys. Exact cumulative token boundaries then assign responses to action events.

Reserve the full requested reply against both context limits and compare estimates with returned usage. Preserve a user query and complete tool relationships when trimming. Charge completed generation before dispatch, including replies whose tool calls will be discarded.

Under atomic charging before dispatch, each positive action-ledger increment equals a contiguous sum of completed-response tokens. Its final response issued the action; preceding responses did not. Remaining responses form the terminal suffix. For example, if successive replies use 10, 20 and 7 tokens and the next action records a charge of 30, the second reply is action-issuing; the first is intervening work. If no later action occurs, the seven tokens form the terminal suffix. This gives a falsifiable partition, not a semantic measure of useful reasoning. An action ledger that omits the suffix understates completed generation; cumulative counters may already retain it.

A generation cap can end at a statement boundary, leaving a parseable prefix. Executing it can spend actions before an intended continuation exists; extra actions penalize a completed level approximately as the inverse square of action count [1]. The guard rejects all cap-flagged calls before dispatch/history insertion, preserves recoverable reasoning, and stops after four consecutive malformed or capped-call replies. It may reject actually complete code; that cost is unmeasured.

Response and dispatch evidence

Table entries are **percentages of completed generation tokens**, not percentages of responses. Terminal takes precedence; other rows partition into action-issuing, non-action tool and no-call responses. Caps overlap these categories. Non-action computation is not necessarily wasted.

Population	Tokens	Action	Other tool	No call	Terminal
R0	17,674,210	45.73	45.07	5.92	3.28
R1	17,048,235	48.72	42.76	5.90	2.62
R2	17,169,163	45.92	45.78	4.90	3.41
Guard qualification	937,189	57.12	27.11	7.87	7.90
Native Franzen	903,752	46.52	47.38	1.36	4.74

R0-R2 each cover 100 public episodes under a nine-hour notebook ceiling; qualification has 20 responding episodes and five unadmitted games under one hour. Native practice covers ten games with 25-minute budgets. Different workloads prevent attributing terminal-share differences to duration. Unanswered requests number 7/8/18/20/5 respectively; these totals exclude unknown work.

Native action history omits 42,811 completed tokens; solver cumulative counters retain them. Completed generation is 595.24 tokens/s over a shared 1,518.31-second gameplay interval, excluding startup. Server billing remains unresolved. Qualification's maximum prompt plus full reply is 130,446, leaving 626 tokens; an 11-token estimation shortfall fit the reserve.

A saved post-hoc estimator candidate reproduced the pinned server's default fields function.strict=false and tool.defer_loading=null, matching returned prompt usage on all 145 sampled requests from two R0 logs. The saved full-R0 audit shows the 11-token shortfall in all 8,840 completed requests; the serializer mechanism was replayed on 145 sampled requests. This identifies the mechanism for that sample, not an exhaustive proof for arbitrary request formats, all 582 qualification responses or later serving versions; the scored estimator remains unchanged. The package includes the saved diagnostic report and its source hashes.

Pre-guard arm	Capped tool replies	Invalid / parseable / empty	Actions	All-cap token share
R0	12	9 / 2 / 1	0	8.00%
R1	8	4 / 4 / 0	1	6.78%
R2	9	7 / 2 / 0	0	6.16%

The 29 payloads consumed 356,352 tokens; all 295 capped replies consumed 3,624,960. These are exposure costs, not measured harm. One parseable R1 call produced a recorded mouse action, corroborated by its tool result; its utility is unknown. Syntax alone cannot certify that a cap-flagged program expresses its complete intended operation. Earlier private mocked replay blocked all 29 saved payloads through the frozen guard, retaining charges and testing continuation. No saved game code executes. Qualification had no naturally capped tool payload; its recovery evidence used fixtures.

Feature comparisons and limitations

Across 25 public games and four episodes per arm, both completed comparisons are reported:

Arm	Mean score	Difference from R0	Adjusted bootstrap interval
R0	44.71	-	-
R1	45.54	+0.83	[-6.85, +8.21]
R2	49.12	+4.41	[-3.23, +11.78]

The paired-game bootstrap uses 12,000 draws, seed 20261024 and alpha=0.05/3, retained from three planned baseline comparisons; R3 was not run. Neither interval excludes zero. Zero observed model-API calls rules out evidence of that intended mechanism, not prompt-mediated effects from exposing the tools.

Verbatim-note retention is a separate five-game screen: two paired blocks, five streams per arm sharing a server, with launch order reversed. The treatment inserts bounded notes containing earlier reasoning at existing history/cache breaks; the control omits those notes. It differs from R1's structured mechanics records and Fang's solved-level memory. Notes-off/on means are 35.36/32.17. Thirteen nonempty flushes exposed notes in eight treatment episodes.

All differences below are computed from unrounded values. Within the same condition, ar25 notes-on scores are 77.78 and 58.33 (19.44 points apart); lp85 notes-off scores are 21.42 and 41.67 (20.25 apart). These spreads exceed the 3.20-point overall arm difference but not the largest paired treatment gap, 41.67. With only two blocks, they are descriptive instability, not a reliable variance estimate. Mean treatment differences also vary by game: ar25 -31.94, lp85 +18.46. Shared serving, game reuse and the small number of independent games prevent a general conclusion about memory.

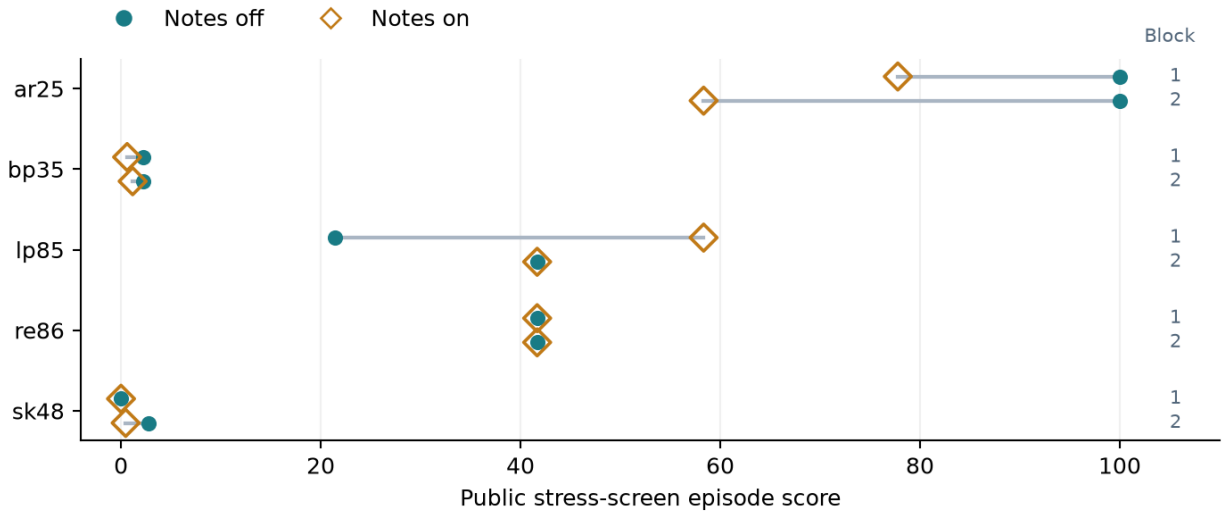


Figure 1. Each connected pair is one game/block, notes off versus on. All 20 episode scores are shown; overlapping equal scores are indicated. These are descriptive observations, not independent game-level replications.

In a separate CPU fixture, four consecutive capped 512-token replies expose another limitation: the submitted guard charges all 2,048 tokens but returns third-reply reasoning instead of fourth-reply reasoning. The package includes a separately labeled post-hoc repair and CPU tests; neither its neural performance nor score effect is known. Frozen submitted source remains unchanged.

Conclusion

Cap-flagged tool dispatch produced one recorded action in 26,484 pre-guard responses (approximately 0.0038%). This establishes rarity in these logs, not negligible damage: the action's score effect is unknown. The broader measured exposure is the 6-8% generation share spent on capped replies; those tokens are not proven wasted, because reasoning may help a later action.

The accounting checks could transfer to other tool-calling agents with per-response usage and finish reasons, recoverable episode/response order, and an action log satisfying the same charge-before-dispatch contract. The dispatch checks also need tool payloads. This applicability is untested beyond this system; asynchronous tools or different accounting semantics need explicit alignment rather than assuming the present reconstruction works unchanged.

Public games informed development, hidden traces are unavailable, and there is no isolated guard ablation. The contribution is reproducible evidence separating completed work, executed actions and feature exposure. It does not establish that the guard improves scores or that the agent reasons or generalizes better.

Availability

The public package, tag paper-audit-v1-20261108, was released November 9, 2026. Evidence-manifest SHA-256: 96c896a46f880119afd8cae611da5ce7ca043122ec0cff2adf04bc357414c197. Original code contributions are MIT-0; the paper is CC-BY 4.0. Upstream licenses remain applicable. The linked public artifact is R2 V1, submission 57684318. Scalar records identify population, episode, response ordinal, completion tokens, class and cap flag, enabling partition recomputation. Full finish reasons, raw logs and saved Python payloads remain private, limiting independent source verification and replay.

References

[1] ARC Prize. Scoring methodology.
[2] Tufa Labs. Duck.
[3] Daniel Franzen. Milestone 2 solution.
[4] Zheng et al. SGLang; Pezzulli. Pennyroyal.
[5] Yao et al. ReAct.
[6] Shinn et al. Reflexion.
[7] Yiming Fang. Lalalia V1.
[8] Kapoor et al. AI Agents That Matter.
[9] Kapoor et al. Holistic Agent Leaderboard.